

In this project, you will program a **shell** in **C/C++**, as these are the traditional programming languages for operating systems. By creating a shell by yourself, you will gain insights in how processes communicate with the operating system and other processes, and how processes are created and controlled.



NOTE

This is a group project (groups of 2).



START TIME

*After you read the specified parts of the **first three chapters** (see weekly exercises) you can start with this project.*



DURATION

*Keep in mind the time you spend on this project: on average it will take **18 hours** per group member. Are you stuck? Please seek assistance.*



OBJECTIVES

- being able to make system calls from your program;
- start processes and let them communicate with each other;
- being able to describe the structure of command prompts;
- gain insight into the workings of operating systems and processes within;
- being able to read the documentation of system calls (using the **man** command);
- getting hands-on experience with an open-source operating system from the UNIX family.



DEADLINE

You should hand it in before Monday 29 September, 23:59. Send it to us using Brightspace.

Background

A **shell** is a text based interface to the operating system. Shells formed one of the first interactive interfaces for modern computers, with at first a line printer instead of a screen for output. Even today, they have a large part in daily operations, as shells allow users to script applications, combine commands, and quickly pass along extra arguments. That is not possible with the same ease and speed in graphical interfaces. You can see examples of such modern command prompts in recent IDE's (such as **Visual Studio Code**) and recent shells (such as the **fish** shell).

The primary function of a **shell** is to let a user enter commands, execute these commands, and return the results to the user. Examples of UNIX commands are **ls** (directory listing), **pwd** (prints the current working directory), and **date** (prints the current date and time). The prompt is also used to pass extra arguments to applications, such as **ls -l** (prints extra information of the entries in the current directory) and **ls [dirname]** (lists the specified directory). Chained execution of commands is also possible so that the output of one application is used as the input to another application. For example, **date** prints the current date while **date | tail -c 5** only prints the last 5 characters (including new lines) of the date (at our system yielding **CEST** with a newline after). To properly handle all these tasks, a shell needs tight integration with the operating system. System calls such as **fork()**, **execvp()**, **pipe()**, **dup2()**, **waitpid()**, **chdir()**, and **getcwd()** facilitate this integration.

Nowadays, **shells** support higher level programming constructs (they are Turing complete). A simple shell (such as the first shells and the shell you are writing in this project) can be constructed with relative ease. A simple shell has the following control flow: (1) print a prompt, (2) read user input, (3) parse the user input, (4) execute the commands while directly writing the output of these commands to the screen.

Specification

Your **shell** will first display the current working directory, followed by the prompt '\$', and then wait for a line of user input (that is terminated with an enter). Your **shell** must process the following input and execute it in the way specified below:

- If there is a single command and that command is equal to **exit** or the standard input of the shell is closed (closing the input stream can be simulated by pressing **Control-D**):
The **shell** will exit.
- If there is a single command, that consists of **cd** with one argument:
The shell changes the current working directory to the specified directory. This is a so-called **internal command**, as the state of the shell process is changed (in step 1, see the man pages of **chdir** and **execvp**).
- If the input contains a command:
The shell will execute the command, and displays its output and error messages on the screen. The command can process direct input from the user. The shell will wait with processing new input until the command completes. If the command does not exist, the shell will display an error and prompt for the next input.
- If the command consists of multiple (chained) commands, separated by '|':
The output of an earlier command is used as input for the next command. Only the output of the last command will be displayed on the screen. All the error messages of all the commands will be directly printed to the screen. Only the first command can process input from the user.
- If the input ends with '&':
The shell executes the input as specified in the other rules, but does not wait for the completion of the command. The commands will be executed in the background (and the shell continues with displaying a prompt and processing input). The output and error messages will continue to be displayed on the screen. The first command can not get direct input from a user (if the input is not redirected from elsewhere, this input channel should be closed).
- If the last command ends with '>' <file>:
The shell executes the input as specified in the other rules, but writes the output of the last command to a file in the current directory, which is named <file>. If this file exists, it will be truncated and overwritten. Of this last command, only the error messages will be displayed on the screen (not the regular output).

- If the first command ends with '`<`' `<file>`:
The shell executes the input as specified in the other rules, but reads the input of the first command from a file named `<file>` in the current working directory.
- In all cases, clear error messages must be given if certain commands can not be executed (not found, parse error, etc.).

Afterwards, the shell will again display the prompt, and the above process will repeat. Upon request, we have a formal EBNF syntax description (default way of specifying the input that is accepted by a program or function) available of this specification.

Method

The **shell** you are required to make, is a process on your system just like all other programs. It executes multiple commands after each other. For each command, new (operating system) processes are started and the shell waits until all of them are finished. The started processes can communicate with each other, and the kernel of the operating system facilitates this communication. To make the communication easier, many operating systems have a general concept of communication channel, called **file descriptors**.

File descriptors are an important concept in operating systems, especially during programming a shell. We are using the **UNIX** variant of this concept, in which file descriptors are the general interface to files, other processes, network resources, hardware, and the user. In the case of our shell, we will also use file descriptor as the interface to the screen. With each process, by default three file descriptors are associated: **stdin** (input, accessible in code as **STDIN_FILENO**), **stdout** (output, accessible in code as **STDOUT_FILENO**), and **stderr** (error messages, accessible in code as **STDERR_FILENO**). Normally, you can read from and write to these file descriptors with the **read()** and **write()** system calls (note: these two system calls are not used for this project). Or use them to set up the communication channels in new processes (that is what this project is about).

The expression **cmd < input-file | cmd2 > output-file** is **valid**. However, there are technical reasons why not all expressions are accepted (see the specification). For example, we have the **invalid** expression **cmd > output-file | cmd2 < inputfile**. This expression is invalid **by design**. The write-ends and the read-ends of pipes can not be used referenced/connected multiple times to duplicate the output/input. This is because the operating system does not support this functionality: if two processes read from the same **pipe()**, then a byte written on that pipe will be given to only one of the processes and never to both. There must be a process in-between that reads the input and writes it to two file descriptors. The UNIX program **tee** does this for output, while the command **cat** can be used to mix two inputs.

When a process duplicates (with the **fork()** system call), both resulting processes will obtain the same file descriptors. The standard input **stdin** is thus shared, likewise for the standard output **stdout**, and the standard error stream **stderr**. By replacing these file descriptors, the process can read/write to a different source/destination. This replacing enables chaining commands and redirecting input and output to files. A communication channel between processes, such as those made by **pipe()**, has write-ends and read-ends. These are also described by file descriptors. The write-ends and read-ends of a communication channel can be duplicated (with **dup2()** or just like all file descriptors with **fork()**), and the command **close()**

closes only one of the write-ends or read-ends. If on one of the write-ends a byte is written, only one of the read-ends receives this byte. Once **all** write-ends are closed, all read-ends get a **end-of-file** signal (the next **read()** system call on that pipe will produce a return code of 0).

The obvious solution for executing chained commands would be the following: wait until the first one is finished, keep its output, and then you start the second command and write the data to this process. This is not an acceptable solution as it will not work for large outputs, with potentially infinite data streams. Instead both commands have to be started simultaneously and the input/output must be passed immediately to the other command (without any buffers nor read/writes inside the shell process). Note that although with pipes it may seem there are not buffers, a pipe itself is a **buffered** inter process communication channel.

Challenge

Your goal is to build the shell as specified above. There are project files handed out that deal with the parsing. Both C, and C++, are allowed. C is the low-level programming language with good interfacing with almost every library, operating system and hardware. You must manage memory on your own in C. The other option is C++, which is the object-oriented version of C. C++ supports you in memory management, as it will take care already of more things (like calling destructors of stack objects). The goal of the project is to understand how to communicate efficiently with the operating systems, not learn C(++). If you are stuck, please seek help immediately.

HINTS

- A code template is available; please use this as a starting point (see step 0).
- Strings in C have the type **char***. You can not compare C-strings using **a == b** (as you are comparing memory addresses instead of contents), you should use **strcmp(a, b) == 0** (**strcmp()** calculates the difference between the strings). To retrieve a C string from a C++ string named **str**, call **str.c_str()**. Or use the C++ string compare (**a == b** if the type of **a** and **b** is **std::string**).
- Useful test commands are: **echo a b c d**, **pwd**, **ls -l**, and **date**. To check the expected behaviour of these commands, please use the regular shell.
- Examples of chained commands are **du -kd1 | sort -n** (sorting all subdirectories based on size) and **find /etc | head -n 10** (displays first ten files in **/etc**). These chained commands are central to the UNIX way: lots of smaller programs that are very efficient in their specific jobs, and which can be easily combined to achieve powerful effects.
- Use the system resources sparsely; the system can run out of resources quite easily.
- When executing chained commands, all commands should be run concurrently. It may seem logical to run the commands in one at a time, but this will result in buffering problems when much data is redirected and processed.

STEP 0 Read the instructions how to set up a C++ development environment (using the **Visual Studio Code** editor), and download the project files at <https://gitlab.science.ru.nl/operatingsystems/project1>. Try to get the project working. Run it. It should give you a prompt, and you can enter a single command which it will execute. Afterwards your shell will terminate (and probably return to the default shell).

STEP 1 To gain insight into the working of the operating system, shells, and processes you can query the manual pages of system calls on every UNIX machine by executing the command `man [system call]`. Look up at least the following commands: `fork()`, `execvp()`, `pipe()`, `waitpid()`, `open()`, `close()`, `dup2()`, `chdir()`, and `exit()`. These are the only system calls you will need (hint: do not use `read()` or `write()`, and do not use `dup()`). Also take a look at the 'related' section in the manual pages, as these could be relevant for your understanding.

STEP 2 Look at the template given. Zoom in on the data structure given below (defined in `shell.cpp`). The commands are executed from this data structure. Take a look at the given implementation skeleton in the project to understand the basic structure.

```
1 struct Command {
2     vector<string> parts = {};
3 };
4
5 struct Expression {
6     vector<Command> commands;
7     string inputFromFile; // if empty input is from keyboard
8     string outputToFile; // if empty output goes to screen
9     bool background = false;
10 }
```

STEP 3 Think about how to execute the data structure. Which system calls do you need? How is the data structure traversed to execute it correctly (write down the case distinctions)? Write this down **briefly**.

HINTS

- the project is not fully specified (think about `cmd | cd /`), so you can basically choose how to handle underspecified cases. Be sure to document it though!
- both the recursive and iterative version can work, most find the iterative version easier. Focus on readability and correctness.
- don't call `dup2()` in the parent process.
- open resources as late as possible, and close them as early as possible.

STEP 4 Implement your solution, one step at a time (in the same order as the specification). A testing framework is included in the template project (look for `shell.test.cpp`) if you want to use unit tests and functional tests during development, which is highly recommended. Document your code.

STEP 5 Describe how you tested your shell (no unit-test framework required for this project). List the tests you have performed. Given your tests, are you sure your shell does not contain any errors? Given **any** tests, can you be sure the shell does not contain any errors?

STEP 6 Make clear your shell does not suffer from an infinite buffer problem (see above).



HAND IN

- your `shell.cpp` file (see step 4), no other files.
- a report adhering to template 1 (see next page). A table of contents or title page is unnecessary.

Note that the points are also in this template, with 50 points for the report, and 50 points for the code. The code is graded on correctness: does it adhere to the specification in this document, and will it handle all the resources properly.



1 Report shell project Operating Systems Concepts

2

3 Names & and student numbers: ...

4

5 Your report should be in .pdf, and it should contain your names and student numbers.

6 In addition, the following sections are required.

7

8 1. Implementation (max 1 A4)

9

10 Describe your implementation. Mention how the code traverses the Expression data structure, clearly
11 indicating when pipe(), fork(), and waitpid() are called (8pt).

12

13 Provide a list of system calls used by your solution and why/how your program uses them (12pt).

14

15 2. Tests (max 1 A4)

16

17 We expect you to test your shell. Unit tests or a testing framework is not required, and it is sufficient
18 if you just try some commands by hand. Mention the commands you tested. One way to do that, is by making
19 a table with the following columns (10pt, 1pt per test, should have proper coverage):

20

21 command | expected result | actual result

22

23 _____

24

25 Given your tests, are you sure your shell does function correctly? Make a distinction between:
26 errors (problems in human), faults (problems in code), failure (problems in execution). (5pt)

27

28 Given _any_ tests, can you be sure your shell does function correctly, and handles all edge cases
29 correctly (such as out of memory)? (5pt)

30

31 3. Infinite buffer problem (max 1/2 A4)

32

33 Explain why infinite buffers are not possible for a communication channel in an operating system, and how an operating

34

35 Does your implementation potentially need an infinite buffer for the communication channels
36 between chained commands? Explain why (not), and why your chained commands will terminate properly
37 (if all the subcommands execute correctly). Does the place of waitpid() have an influence on this? (5pt)